Week 7: Diffusion and flow matching

Jin-Guo Liu, Xi-Wei Pan

Abstract: We learn how diffusion and flow matching generate data from Gaussian noise. We begin with four groups of points, called clusters. For diffusion, we train a network to predict noise added to the points. For flow matching, we train a network to predict motion between data and noise. We compare how many steps the sampling methods need. Finally, we use a model already trained on photographs to generate new images and fill missing pixels.

Diffusion models

Generating data from Gaussian noise. Week 6 generated samples from an energy model without computing its normalizer Z. This week learns to transform fresh random numbers into data. A *generative model* learns a rule for drawing new examples; drawing an example is *sampling*, and the result is a *sample*. We use a *neural network*, a function with adjustable parameters. *Training* fits those parameters to examples.

Gaussian noise as the starting distribution. We start generation with independent draws from the *standard Gaussian* $\mathcal{N}(0, 1)$, centred at zero with *variance* one (its average squared distance from zero is one). We draw coordinates *independently*, so knowing one draw does not change another's probabilities. Noise is an added random disturbance. A *clean sample* x_0 is data before adding noise. Its coordinates may store a point's position or an image's pixel colours.

Forward and reverse diffusion. The fixed process called forward diffusion repeatedly shrinks x_0 toward zero and adds Gaussian noise. Write the current *state* as x_t after t steps. The step index is the noise level, also called the diffusion level here the final level is $T = 1000$. In Figure 1, read the top row left to right to see data become noise. Generation follows the *reverse process*, reading the lower rows right to left.

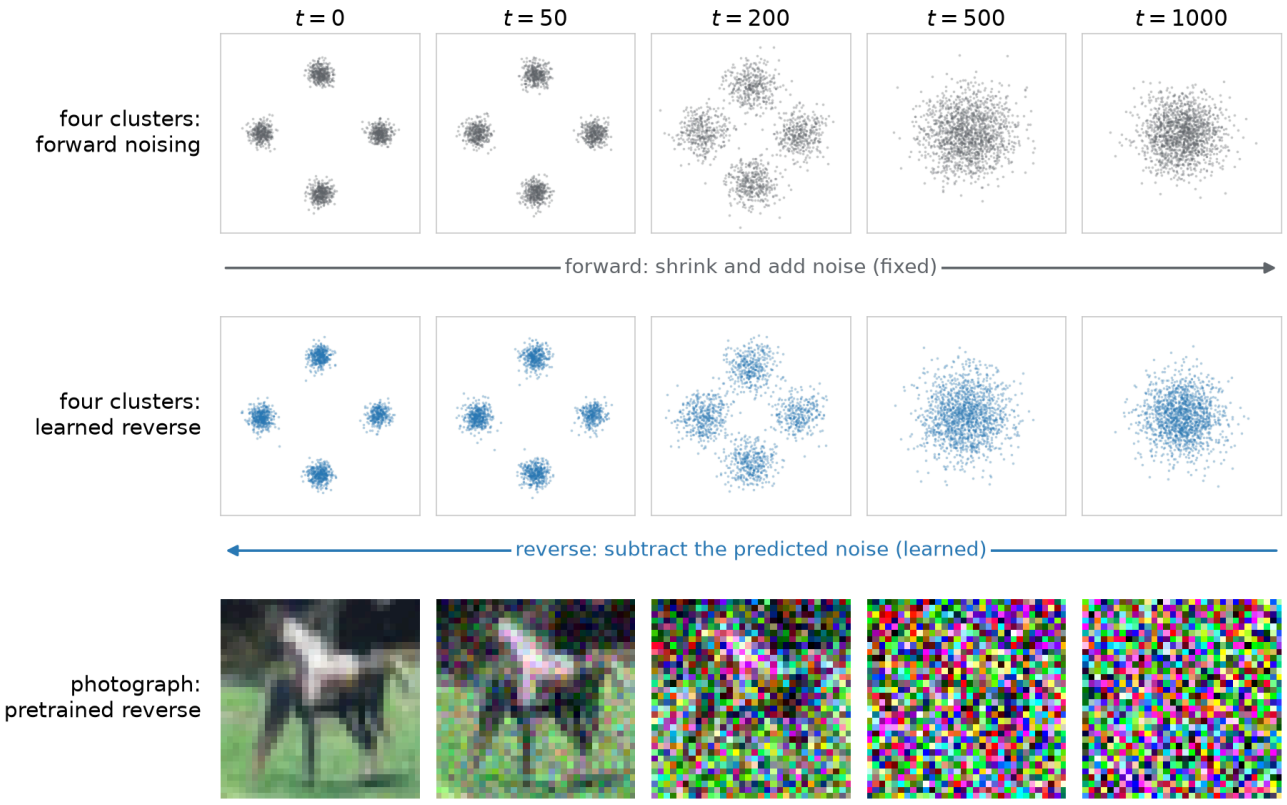


Figure 1: The forward and reverse processes. Top: forward diffusion turns four clusters into an approximately Gaussian cloud. Middle: a learned reverse process turns fresh Gaussian noise back into four clusters. Bottom: a downloaded network guides the reverse process for a 32×32 photograph. Columns use the noise levels defined above. All panels are reference runs from Parts 1 and 4.

Generation by small reverse steps. Starting from fresh Gaussian noise at $t = T$, a learned rule repeatedly moves to a lower level until it reaches $t = 0$. The positions form a *path* or *trajectory*; the first and last are its *endpoints*. The aim

is to recover the data distribution: which outcomes are common and which are rare. It does not recover the particular sample that began a forward run. Estimating a clean sample from a noisy one is *denoising*.

Noise prediction and DDPM sampling. During training, we add known Gaussian noise to clean data. A noise predictor receives the noisy sample and its noise level. It learns to estimate the added noise, the training *target*. Predicting numerical targets this way is *regression*.

After training, a *denoising diffusion probabilistic model* (DDPM) uses these estimates to move a sample to a lower noise level. It also adds fresh noise at every step except the last. Repeating these steps from an initial Gaussian draw produces a new data sample. The rule for these steps is the *sampler*.

Two routes to fewer sampling steps. Each sampling step evaluates a network. Fewer steps reduce the cost of generation. Part 2 uses a *denoising diffusion implicit model* (DDIM) to reuse the trained noise predictor and jump between selected noise levels. Its deterministic update makes no random choices after the initial Gaussian draw. DDPM's steps add fresh randomness and are called *stochastic*. Part 3 uses flow matching to train a new network to predict *velocity*: a point's speed and direction of motion. Its sampler uses these predictions to move points from Gaussian noise toward data. We compare how many steps each sampler needs to recover the four clusters.

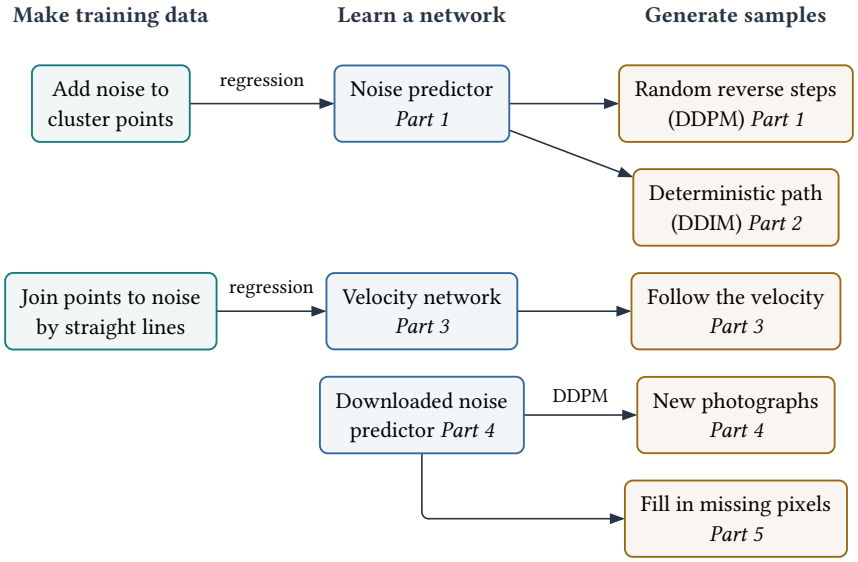


Figure 2: Which network each Part trains or reuses, and which sampler uses it. Parts 1 to 3 use the four-cluster test distribution; Parts 4 and 5 use photographs.

Resources:

- Welch Labs, *But how do AI images and videos actually work?*. Watch for how diffusion models turn noise into images.
- Niels Rogge and Kashif Rasul, *The Annotated Diffusion Model*. Parts 1 and 4 follow its noising, noise-prediction and reverse-sampling sequence. Our implementation uses JAX.
- Ho, Jain and Abbeel, *Denoising Diffusion Probabilistic Models*. Algorithms 1–2 are the training and sampling loops of Parts 1 and 4.

Part 1: Diffusion models on a Gaussian mixture

Goal: train a noise predictor and use it to generate new samples. Check that the samples form the four target clusters, with the right fraction of points in each cluster and the right centres and widths.

① DO

Work in `week7/` inside your existing exercise repository. Your laptop's central processing unit (CPU) is the baseline. Send the prompts, then read UNDERSTAND while your agent works.

```
Work in week7/ using uv, with CPU as the default. Save figures in evidence/ and raw
runs with settings in artifacts/. Keep downloads and raw runs out of git. Commit after
each task; ask about uncertainties instead of guessing.
```

Next, build the four-cluster target and the process that adds noise to it.

```
Implement only target sampling and forward diffusion in JAX. The target is an equally
weighted Gaussian mixture with centres (3,0), (0,3), (-3,0), (0,-3) and covariance
0.09 times the identity. Use 1000 steps with per-step noise variance beta_t increasing
linearly from 0.0001 to 0.02, where t is the diffusion level.
Do not train a network or implement reverse sampling in this task.
```

Finally, train the noise predictor and use it to generate new points. Save this block as `week7/diffusion.toml` in your exercise repository.

diffusion.toml

```toml
[model]
hidden_widths        = [128, 128, 128]
activation           = "SiLU" # u / (1 + exp(-u)), also called swish
time_features        = 32     # 16 sine and 16 cosine features
time_frequency_base  = 10_000.0
time_offset          = -1     # Features use t - 1 for levels t = 1,...,1000
# For j = 0,...,15, use sine and cosine of
# (t + time_offset) * time_frequency_base ** (-j / 16).
weight_initialization = "normal with std 1/sqrt(input width)"
bias_initialization  = 0.0

[training]
updates              = 25_000
batch_size           = 512    # Fresh target samples at every update
optimizer            = "Adam"
beta1                = 0.9
beta2                = 0.999
epsilon              = 1e-8
learning_rate        = 0.001
learning_rate_decay  = "cosine"
final_learning_rate  = 0.0
precision            = "float32"
seed                 = 2026
checkpoint_updates   = [0, 50, 100, 200, 500, 1000, 2000, 5000, 10000, 25000]
```

```
Train a small network to predict the added noise using `diffusion.toml`. Make the
predictor time-conditioned. Minimize mean squared noise-prediction error over
uniformly sampled noise levels. Record training settings and loss history. Retain the
model at the specified checkpoint updates, including initialization before training.
After training, implement DDPM sampling using the trained noise predictor. Use
reverse-step variance beta_t, no clipping and no added noise in the final step.
```

The reference completed this training in about three minutes on two CPU cores.

(2) UNDERSTAND

Drawing the four-cluster target. Our sample is a pair of real numbers, a *vector* $x \in \mathbb{R}^2$. Choosing among Gaussian clusters gives a Gaussian mixture. Each cluster is a *component*. Our four centres are

$$\mu_1 = (3,0), \quad \mu_2 = (0,3), \quad \mu_3 = (-3,0), \quad \mu_4 = (0,-3). \tag{1}$$

Give each component index $k \in \{1,2,3,4\}$ equal probability and draw independent Gaussian numbers z_1, z_2, independently of k. Then

$$x_0 = \mu_k + 0.3(z_1, z_2), \quad z_1, z_2 \sim \mathcal{N}(0,1) \ \text{ independently.} \tag{2}$$

These are *exact samples*, drawn directly from the prescribed target. Parts 1 to 3 use them to train and to check learned samplers.

Describing the target with a density and averages. A *probability density* $p(x)$ integrates to a region's probability and to one over the whole space. Its value at one point is not a probability. An *expected value* is a density-weighted average:

$$\mathbb{E}[h(x)] = \int h(x)p(x)\,\mathrm{d}x. \tag{3}$$

A sample average estimates this integral. The mean $\mu = \mathbb{E}[x]$ averages each coordinate separately.

Covariance and cluster width. The *covariance matrix* records the spread:

$$\Sigma_{ij} = \mathbb{E}\big[(x_i - \mu_i)(x_j - \mu_j)\big]. \tag{4}$$

Its diagonal entries are coordinate variances; off-diagonal entries describe how coordinates vary together. The square root of a variance is the *standard deviation*. Each component above has independent coordinates of standard deviation $\sigma = 0.3$, hence covariance $0.09I$. The *identity matrix* I has ones on its diagonal and zeros elsewhere.

The target density. Write $\mathcal{N}(x; \mu, \Sigma)$ for the Gaussian density with mean μ and covariance Σ. The recipe gives

$$p_{\text{data}}(x) = \frac{1}{4}\sum_{k=1}^{4} \mathcal{N}(x; \mu_k, 0.09I). \tag{5}$$

Its four separated regions of high density are the target's modes. The notation $z \sim \mathcal{N}(0, I)$ will mean a vector of independent standard Gaussian draws, each with mean zero and variance one.

Adding noise with the prescribed variance schedule. The forward update shrinks a point and adds independent noise. The diffusion level runs through $t \in \{1, \ldots, T\}$, with $T = 1000$. Let β_t be the variance of the noise added to each coordinate in step t, and set $\alpha_t = 1 - \beta_t$. The update is

$$x_t = \sqrt{\alpha_t}\,x_{t-1} + \sqrt{\beta_t}\,z, \quad z \sim \mathcal{N}(0, I). \tag{6}$$

The *variance schedule* $\beta_1, \ldots, \beta_T$ grows linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$, so early steps add less noise than later ones. The shrinkage multiplies the previous variance by α_t; independent noise adds β_t. Since $\alpha_t + \beta_t = 1$, variance one stays one.

Direct noising to any level. To construct a training example at level t, combine the preceding Gaussian disturbances into one draw. Define $\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\overline{\alpha}_0 = 1$. The result of the t steps has the same distribution as

$$x_t = \sqrt{\overline{\alpha}_t}\,x_0 + \sqrt{1 - \overline{\alpha}_t}\,\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \tag{7}$$

Here ε is the known noise for this training example. At $t = 1000$, $\overline{\alpha}_t \approx 4 \times 10^{-5}$, so the clean point contributes very little and the state is approximately standard Gaussian.

Predicting noise from the point and its level. The input is (x_t, t) and the output $\varepsilon_\theta(x_t, t)$ has two coordinates, one estimated noise value per coordinate of the point. The parameters are θ. Supplying the noise level makes the network time-conditioned the index t is also called *time*. The same point can require different predictions at different noise levels, so its coordinates alone do not specify the prediction task. The clean point x_0 is unavailable during generation and is never an input.

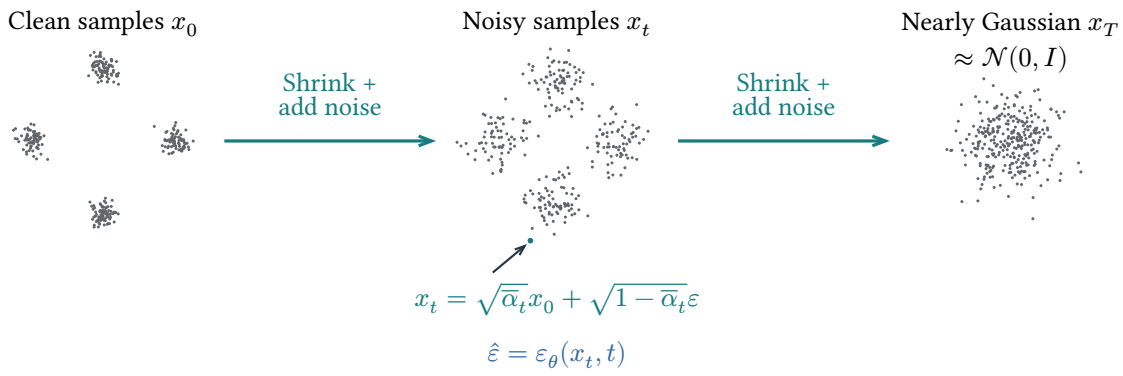


Figure 3: Training inputs at three noise levels, using the same 300 reference samples. The arrow follows one sample. Teal constructs the noisy input from known noise; blue predicts that noise from the input and its level.

Encoding the noise level as network inputs. The network specified in `diffusion.toml` converts the integer $t \in \{1, \ldots, 1000\}$ into *time features* using 16 fixed frequencies:

$$\omega_j = 10000^{-\frac{j}{16}}, \quad j = 0, \ldots, 15. \tag{8}$$

Its time-feature vector is

$$r(t) = [\sin((t-1)\omega_0), \ldots, \sin((t-1)\omega_{15}), \cos((t-1)\omega_0), \ldots, \cos((t-1)\omega_{15})] \in \mathbb{R}^{32}. \tag{9}$$

The shift $t - 1$ is the reference implementation's indexing convention. Low frequencies vary slowly with the level; high frequencies vary more rapidly. Append these 32 numbers to the point's two coordinates:

$$h_0 = [x_t; r(t)] \in \mathbb{R}^{34}. \tag{10}$$

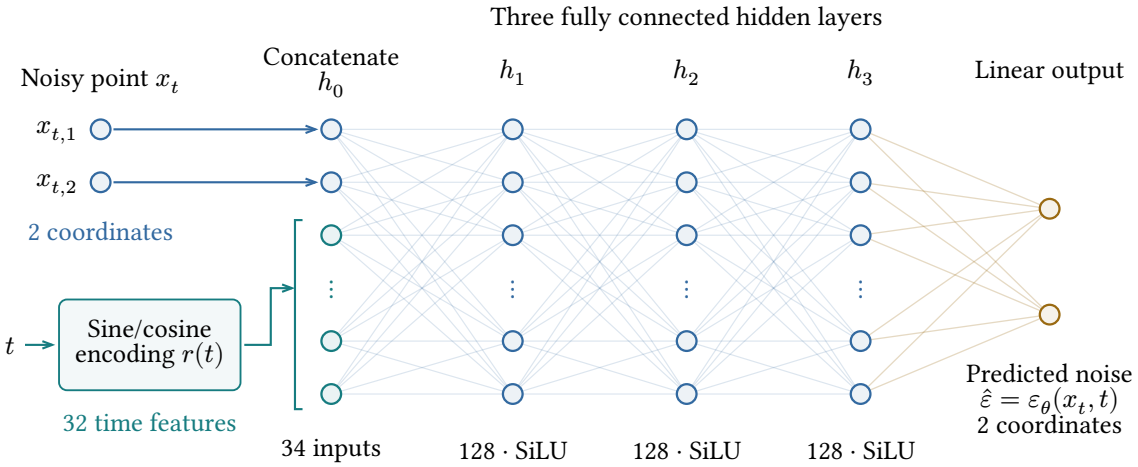
The semicolon means joining the two lists of numbers. Every example supplies its own noisy point and level. Training and sampling use the same fixed map r; the network learns how to use these inputs.

The predictor's architecture and weights. Three *hidden layers* of 128 values each, intermediate vectors h_1, h_2, h_3, then compute

$$h_l = \varphi(W_l h_{l-1} + b_l), \quad l = 1, 2, 3, \quad \varepsilon_\theta(x_t, t) = W_4 h_3 + b_4. \tag{11}$$

Each matrix W_l maps the preceding vector to the next layer's size, and b_l is an added vector, called a *bias*. Every output of this matrix multiplication can depend on every input, so the layer is *fully connected*. The function $\varphi(u) = \frac{u}{1+\exp(-u)}$, applied to each coordinate, is the *activation*, called SiLU or swish; its nonlinear response prevents the layers from collapsing into one matrix multiplication. The final layer returns two noise coordinates without an activation.

This arrangement is the network's architecture. Its adjustable weights, or *parameters*, are $\theta = \{W_l, b_l\}_{l=1}^4$. *Initialization* assigns their starting values. The file specifies zero biases and independent normal weights with standard deviation $\frac{1}{\sqrt{n}}$, where n is the number of inputs to that layer. Training changes θ while keeping the time features and architecture fixed.



Trained weights θ stay fixed; the inputs x_t and t change during sampling.

Figure 4: The trained noise predictor. The noisy point and encoded noise level feed three fully connected SiLU layers, followed by a linear output with two noise coordinates. Circles represent selected entries of each layer; labels give the full sizes. Sampling changes the point and level while keeping the trained weights fixed.

The noise-prediction loss. To judge a prediction, subtract the known noise and square each coordinate's error. For a vector w, its length, or *norm*, is $\|w\| = \sqrt{\sum_i w_i^2}$. Thus $\|w\|^2$ is the sum of its squared coordinates. The mean squared noise-prediction error is

$$\mathcal{L}_{\text{noise}}(\theta) = \mathbb{E}_{x_0, t, \varepsilon}\left[\frac{1}{2}\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2\right]. \tag{12}$$

The expectation averages over clean points, noise levels and Gaussian noise. We draw levels *uniformly*, giving every level the same probability. The factor $\frac{1}{2}$ averages over the two coordinates. The algorithms use d for the coordinate count, so here $d = 2$. This quantity is the training *loss*, the number we try to reduce.

Updating the weights during training. A *batch* is a group of training examples. Its average loss is $\hat{\mathcal{L}}$. The *gradient* $\nabla_\theta \hat{\mathcal{L}}$ collects the derivatives with respect to the weights; JAX computes it by automatic differentiation, as in Week 5. An *optimizer* uses this gradient to update the weights. For example, gradient descent uses $\theta \leftarrow \theta - \eta_j \nabla_\theta \hat{\mathcal{L}}$, where the *learning rate* η_j sets the step size at update j. Decaying η_j toward zero makes late updates smaller. An optimizer may also keep a state q, such as running gradient averages. Write $U(\theta, q, g, \eta)$ for its supplied update rule, returning the next weights and state from gradient g and learning rate η. The file selects Adam, which uses running averages of gradients and their squares to scale the updates. Its two averaging factors are `beta1` and `beta2`; `epsilon` keeps the denominator positive. Its cosine learning-rate decay decreases smoothly from the initial rate to zero over the specified updates. The batch size is the number of fresh points per update; `float32` stores numbers with 32 binary digits. The seed fixes the sequence of random draws. A saved set of weights is a *training stage*; `checkpoint_updates` lists when to save them, with zero denoting initialization. All sampling statements below use fresh independent draws; Uniform denotes the uniform distribution.

Algorithm 1. Train the noise predictor

Input: Predictor ε_θ with initial weights θ; data distribution p_{data} on $\mathbb{R}^d$.
Noise schedule $\beta_1, \ldots, \beta_T$; batch size B; number of updates J.
Optimizer U with initial state q; learning rates $\eta_1, \ldots, \eta_J$.
Output: Trained predictor ε_θ and fixed coefficients $\alpha_t, \overline{\alpha}_t$ for sampling.

▷ *Set up the fixed noise schedule.*
$\alpha_t \leftarrow 1 - \beta_t \quad (t = 1, \ldots, T)$
$\overline{\alpha}_0 \leftarrow 1$
$\overline{\alpha}_t \leftarrow \prod_{k=1}^{t} \alpha_k \quad (t = 1, \ldots, T)$
for $j = 1, \ldots, J$ (training updates) **do**
  $g \leftarrow 0$ ▷ *Start a new batch gradient.*
  for $i = 1, \ldots, B$ (examples in this batch) **do**
    $x_0 \sim p_{\text{data}}$ ▷ *Clean sample.*
    $\varepsilon \sim \mathcal{N}(0, I)$ ▷ *Independent noise.*
    $t \sim \text{Uniform}(\{1, \ldots, T\})$ ▷ *Noise level.*
    ▷ *Construct the noisy input using Equation 7.*
    $x_t \leftarrow \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1 - \overline{\alpha}_t}\varepsilon$
    $\ell(\theta) \leftarrow \frac{1}{d}\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2$ ▷ *Noise-prediction loss.*
    $g \leftarrow g + \frac{1}{B}\nabla_\theta \ell(\theta)$ ▷ *Add this example's contribution.*
  end
  ▷ *Update the weights once, after all B examples.*
  $(\theta, q) \leftarrow U(\theta, q, g, \eta_j)$
end
return $\varepsilon_\theta, \{\alpha_t\}_{t=1}^T, \{\overline{\alpha}_t\}_{t=0}^T$

DDPM: generating samples by removing noise. Keep the trained network fixed. In Algorithm 2, x holds the current state. Line 4 converts the predicted noise into the mean previous state. Line 9 samples around that mean; the random term supplies the spread. Lines 5–8 omit this randomness at the final step.

Algorithm 2. DDPM sampling

Input: Fixed noise predictor ε_θ and scalar coefficients $\alpha_t, \overline{\alpha}_t$ returned by training; noise variances $\beta_1, \ldots, \beta_T$.
Output: Generated clean sample x.

$x \sim \mathcal{N}(0, I)$ ▷ *Initial state at level T.*
for $t = T, T-1, \ldots, 1$ **do**
  $\hat{\varepsilon} \leftarrow \varepsilon_\theta(x, t)$
  $\mu \leftarrow \frac{1}{\sqrt{\alpha_t}}\left(x - \frac{\beta_t}{\sqrt{1 - \overline{\alpha}_t}}\hat{\varepsilon}\right)$
  $z \leftarrow 0$
  if $t > 1$ **then**
    $z \sim \mathcal{N}(0, I)$
  end
  $x \leftarrow \mu + \sqrt{\beta_t} z$ ▷ *Advance to level $t - 1$.*
end

11: **return** x

The two means in DDPM sampling. A *conditional density* $p(u \mid y)$ describes an unknown u after observing y; the bar means "given". Its probability-weighted average is the conditional mean, $\mathbb{E}[u \mid y]$. Squared-error training makes Algorithm 2, line 3 estimate the conditional mean noise. Line 4 converts it into an estimate of the conditional mean previous state. With an exact noise conditional mean, the previous-state mean would also be exact.

The variance of a reverse step. Returning only the mean would discard the spread of possible previous states. Let p_t be the density at level t and $H_t(y) = \nabla_y^2 \log p_t(y)$ its log-density's matrix of second derivatives. Differentiating the Gaussian forward density gives the exact conditional-covariance identity, Eq. S33

$$\mathrm{Cov}(x_{t-1} \mid x_t = y) = \frac{\beta_t}{\alpha_t}(I + \beta_t H_t(y)). \tag{13}$$

If $H_t(y)$ stays bounded as β_t tends to zero, this covariance is $\beta_t I + O(\beta_t^2)$. Algorithm 2, line 9 uses the leading term in a Gaussian approximation. Its factor $\sqrt{\beta_t}$ is the standard deviation; the variance is β_t. This choice approximates the reverse spread separately from learning its mean.

Why the mixture sampler must not clip to $[-1, 1]$. Replacing values below a lower bound by that bound, and values above an upper bound by that bound, is clipping. Clipping to $[-1, 1]$ would exclude this mixture's four centres, so we omit it.

KEY CONCEPTS
- **Target:** equal cluster populations, prescribed centres and coordinate spread 0.3.
- **Forward diffusion:** shrink and add Gaussian noise; the schedule sets its variance.
- **Time conditioning:** append fixed sine and cosine features of the noise level to the noisy point's coordinates.
- **Learning:** squared noise error trains the predictor toward the conditional mean noise.
- **Generation:** DDPM uses the fixed predictor for reverse steps, with no added noise at the last step.

3 VERIFY

(1) **Forward diffusion turns four clusters into one Gaussian.**

```
Make a movie of forward diffusion showing how the modes merge. Follow the same
samples as the noise level increases, with a fixed coordinate scale. Print each
coordinate's mean and variance at t=1000.
```

The centres move toward the origin while each cloud broadens; by $t = 1000$ the four groups are indistinguishable. With at least 2000 samples, each coordinate's mean should lie within 0.1 of zero and its variance within 0.1 of one. Watch the reference forward process.

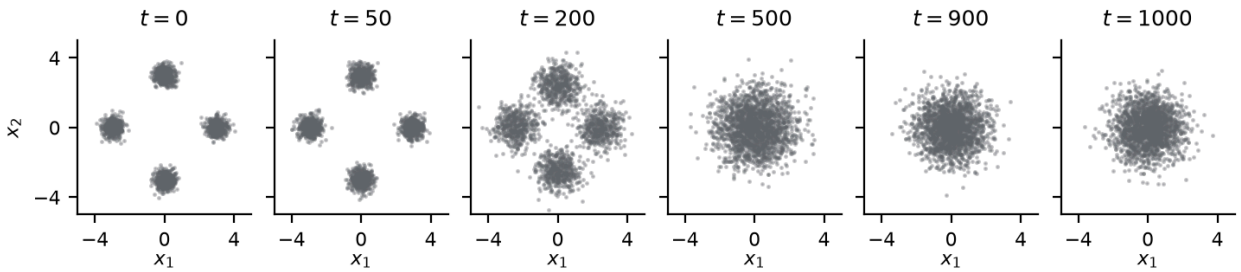


Figure 5: The forward end state is approximately standard Gaussian. The reference follows the same 2000 points through forward steps at six levels on one scale.

(2) **Training moves the generated endpoints onto the clusters.**

```
Make a movie using the predictors saved at training updates 0, 50, 100, 200, 500,
1000, 2000, 5000, 10000 and 25000. At each stage, show 10 DDPM generation paths and
```

```
the training-loss history so far. Label each stage with its training update number.
Reuse the same initial Gaussian samples and reverse-step noise across stages.
```

Each path runs from $t = 1000$ to 0 with fixed weights; the training stage changes the weights. Shared starts and step noise isolate the effect of learning. At initialization the paths leave the plotted square, some reaching $|x| \approx 1500$; at the final stage all ten endpoints sit on the four clusters. In the reference most of the change happens in the first few hundred of its 25000 updates, while the loss falls.

A positive training loss can remain after the clusters emerge. The reference loss levels off near 0.25. A positive plateau alone does not establish failed training: different clean points and noise draws can produce the same noisy input. For example, at the origin all four clusters remain possible. The predictor must average over those possibilities. At a fixed level t, write $m = \mathbb{E}[\varepsilon \mid x_t]$ and let a be any prediction from x_t. The squared error separates exactly as

$$\mathbb{E}[\|\varepsilon - a\|^2 \mid x_t] = \mathbb{E}[\|\varepsilon - m\|^2 \mid x_t] + \|m - a\|^2. \tag{14}$$

Only the second term can be reduced by changing the prediction. The first is uncertainty about the noise that remains even at the best prediction $a = m$. This explains why zero loss is not the target; it does not prove that the observed plateau is optimal. Judge the generated distribution using the cluster populations, centres and widths in the next check. Watch the reference training process.

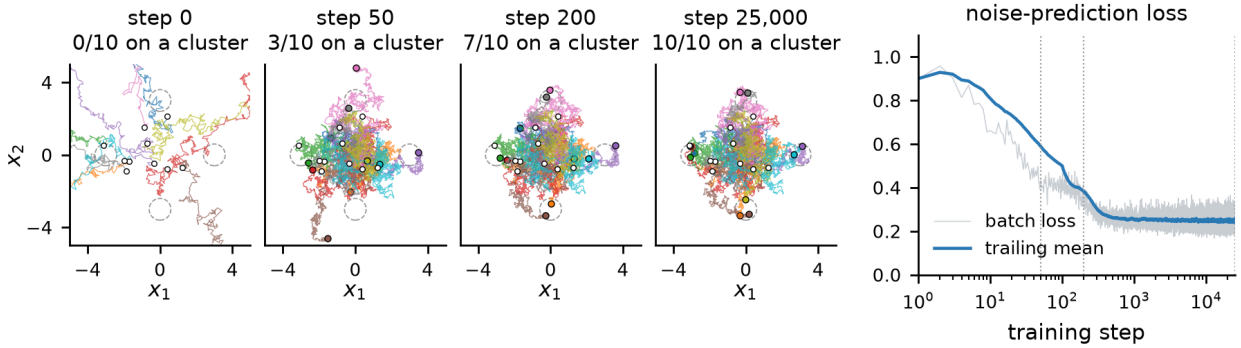


Figure 6: Endpoints move onto the four clusters as the loss falls. The same ten generation paths (white starts, coloured ends, dashed circles of radius 2σ around the centres) with the predictor saved at four of its ten training stages; dotted lines mark those stages on the loss history.

(3) **Generated samples recover the cluster populations, centres and widths.**

```
Generate 2000 samples with the trained predictor and 1000 DDPM steps. Make a movie
of the reverse process and compare the final distribution with the target. Plot the
samples and measure each mode's centre, population and per-coordinate spread.
```

Assign each generated point to its nearest target centre. Each group's *population* is its fraction of all points; its *per-coordinate spread* is the standard deviation in each coordinate. Your populations should lie within 0.04 of 0.25, your spreads within 0.03 of 0.30 and your centres within 0.05 of the targets. The reference gives populations from 0.245 to 0.262 and spreads from 0.29 to 0.30. All three measurements are needed: correct populations can still hide shifted or broadened clusters. Watch the reference generation process.

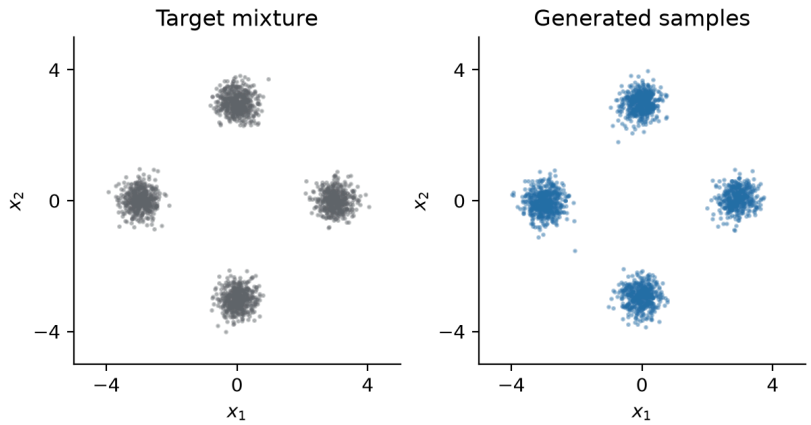


Figure 7: Generated samples recover the target's centres, equal populations and spread. The reference run's samples are plotted beside exact target samples.

Part 2: Deterministic sampling with DDIM

Goal: replace DDPM's random reverse step with a deterministic update that reuses the trained noise predictor and can jump between selected noise levels. Check that repeated runs from one starting noise agree exactly, and that the generated points still recover the four clusters.

1 DO

```
Implement deterministic DDIM sampling with the trained noise predictor over a
selectable subset of noise levels. Also support the mixture's exact conditional-mean
noise prediction for comparison.
```

2 UNDERSTAND

Deterministic generation with the same trained predictor. All the randomness can be chosen at the start. The same trained noise predictor then guides a deterministic path to a sample, with no retraining. To see how, think of the current noisy point as a combination of clean data and noise. The predictor lets us estimate these two contributions. DDIM changes their proportions to form a less noisy point, then repeats the prediction there.

Algorithm 3 visits a decreasing list of noise levels, the *grid*. Its S jumps require S predictor evaluations. The coefficients come from Algorithm 1; choosing fewer sampling levels does not change the training noise schedule.

Algorithm 3. Deterministic DDIM sampling

Input: Fixed noise predictor ε_θ and coefficients $\{\overline{\alpha}_t\}_{t=0}^{T}$ returned by Algorithm 1.
Decreasing levels $T = t_0 > t_1 > \ldots > t_S = 0$.
Output: Generated sample x_0.

1: $x_T \sim \mathcal{N}(0, I)$ ▷ *The only random draw.*
2: **for** $k = 0, \ldots, S-1$ **do**
3: $(t, t') \leftarrow (t_k, t_{k+1})$ ▷ *Current and next levels.*
4: $\hat{\varepsilon} \leftarrow \varepsilon_\theta(x_t, t)$ ▷ *Predict noise at the current state.*
5: $\hat{x}_0 \leftarrow \dfrac{x_t - \sqrt{1-\overline{\alpha}_t}\,\hat{\varepsilon}}{\sqrt{\overline{\alpha}_t}}$ ▷ *Estimate clean data.*
6: $x_{t'} \leftarrow \sqrt{\overline{\alpha}_{t'}}\,\hat{x}_0 + \sqrt{1-\overline{\alpha}_{t'}}\,\hat{\varepsilon}$ ▷ *Move to level t'.*
7: **end**
8: **return** x_0

Lines 3–4 supply the original noise level. The loop counter k counts sampling jumps. Line 3 selects the noise level $t = t_k$ from the grid, and line 4 supplies that level to the predictor using Part 1's encoding $r(t)$. For a grid beginning $1000, 800, 600, \ldots$, jump $k = 1$ starts at level 800: the network receives the current point with $r(800)$, and the update uses the stored coefficient $\overline{\alpha}_{800}$. Skipping levels changes which predictions are evaluated; their inputs retain the meaning used during training.

Line 5 estimates the clean point. Line 4 predicts the noise in the current point. Line 5 subtracts its contribution, then undoes the shrinkage of the clean point in Equation 7. A function that estimates clean data this way is a *denoiser*. With

the exact conditional-mean noise, this is *Tweedie*'s formula: the estimate is $\mathbb{E}[x_0 \mid x_t]$, the average of the clean points compatible with the current noisy point.

Line 6 constructs a less noisy point. Keep the clean estimate and the predicted noise from this step. Combine them in the proportions prescribed by the next level t': a larger clean contribution and a smaller noise contribution. This makes the next state without drawing fresh noise. The next pass through line 4 uses that new state to make a new prediction; the clean estimate can therefore change along the path. At the final level $t' = 0$, the noise contribution vanishes and line 6 returns the clean estimate. See Song, Meng and Ermon, *DDIM*, Section 4, for the construction and sampling on shorter grids.

Replacing line 4 by the exact noise prediction. The comparison also uses $\varepsilon_{\text{exact}}(x_t, t) = \mathbb{E}[\varepsilon \mid x_t]$. Calculate the conditional mean within each Gaussian component, then average using the components' probabilities given x_t. This removes predictor error while keeping the same grid.

KEY CONCEPTS

- **DDIM:** reuse the noise predictor with a deterministic update between selected levels.
- **Noise level:** t_k selects the original schedule coefficient and the network's time features; k counts sampling jumps.
- **Denoiser:** the clean estimate targets the conditional mean; the grid controls step size.
- **Exact predictor:** removes network error so the remaining discrepancy tests the sampling steps.

③ VERIFY

(1) **DDIM's reverse step is deterministic; DDPM's is not.**

```
Run DDIM over 1000 levels twice from the same noise, and DDPM from the same
starting noise with two noise draws. Plot their trajectories and 2000 samples of
each beside exact samples. Print the largest difference between the two DDIM runs.
```

The two DDIM runs agree exactly; the reference prints 0. Algorithm 3 draws randomness only at line 1, so fixing that draw fixes the entire path. Different starting draws still produce different samples. Across the 2000 shared starts, the two DDPM runs end in the same cluster only about a quarter of the time, as chance predicts for four clusters. Both samplers still reproduce the target: DDIM cluster fractions lie within 0.015 of 0.25, and both samplers' widths lie between 0.29 and 0.32.

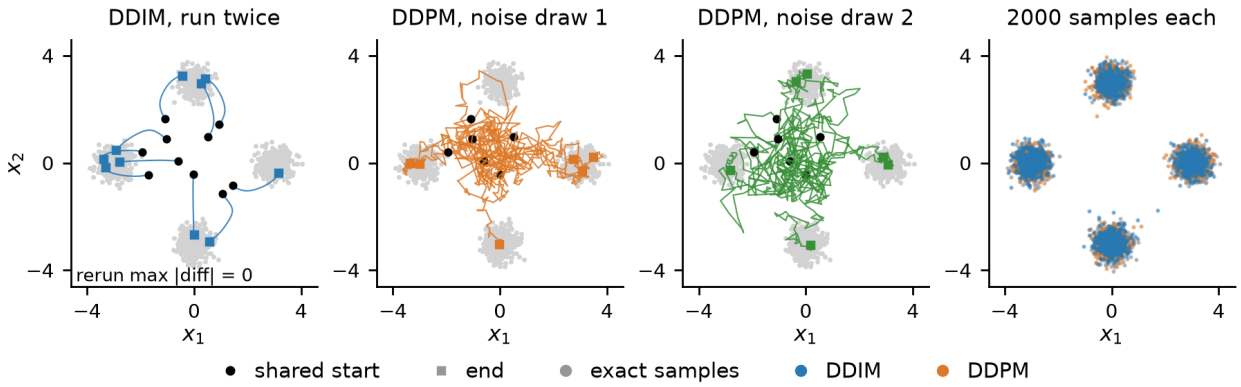


Figure 8: One starting noise gives one deterministic sample; the stochastic sampler does not, yet both reproduce the target distribution.

Part 3: Flow matching and the cost of fewer steps

Goal: train a velocity network on straight paths by flow matching, check it against the mixture's exact marginal velocity, and compare DDPM, DDIM and flow matching on one step grid. Measure how many steps each sampler needs to reach the target.

① DO

Save this block as `week7/flow.toml` in your exercise repository.

flow.toml

```
[model]
hidden_widths        = [128, 128, 128]
activation           = "SiLU" # u / (1 + exp(-u)), also called swish
time_features        = 32     # 16 sine and 16 cosine features
time_frequency_base  = 10_000.0
time_scale           = 1000.0 # Features use 1000 * s, with s in [0,1]
# For j = 0,...,15, use sine and cosine of
# time_scale * s * time_frequency_base ** (-j / 16).
weight_initialization = "normal with std 1/sqrt(input width)"
bias_initialization  = 0.0

[training]
updates              = 25_000
batch_size           = 512    # Fresh target samples at every update
optimizer            = "Adam"
beta1                = 0.9
beta2                = 0.999
epsilon              = 1e-8
learning_rate        = 0.001
learning_rate_decay  = "cosine"
final_learning_rate  = 0.0
precision            = "float32"
seed                 = 2029
```

```
Train a velocity network from scratch on the same mixture by flow matching using
`flow.toml`, with the path x_s=(1-s)x_0+s z, z standard Gaussian, s uniform in [0,1],
and target velocity z-x_0. Sample by Euler steps from s=1 to s=0. Compute the exact
marginal velocity of the mixture for comparison.
```

```
Compare DDPM, DDIM and flow-matching samples with 1,2,5,10,20,50,100 steps, and 1000
levels for DDPM and DDIM. Fewer steps use evenly spaced levels. Use 4000 samples per
run. Record cluster fractions, centres and widths, and the energy distance to 4000
exact samples. Estimate its null scale from 20 pairs of independent exact sample sets.
```

2 UNDERSTAND

Learning motion from data and noise. To generate a sample by moving a noise point, we need a direction and speed at each position and time. Flow matching provides training examples by joining data points to independent Gaussian points with straight paths. Each path has a known velocity. Algorithm 4 trains a network on these velocities using positions chosen along the paths. Time s runs continuously from data at 0 to noise at 1; generation will follow the learned motion backward.

Algorithm 4. Train the velocity network

Input: Velocity predictor v_θ with newly initialized weights θ; data distribution p_{data} on $\mathbb{R}^d$.
Batch size B; number of updates J; optimizer U with initial state q;
learning rates $\eta_1, \ldots, \eta_J$ decaying toward zero.
Output: Trained velocity predictor v_θ.

for $j = 1, \ldots, J$ (training updates) **do**
  $g \leftarrow 0$ ▷ *Start a new batch gradient.*
  for $i = 1, \ldots, B$ (examples in this batch) **do**
    $x_0 \sim p_{\text{data}}$ ▷ *Data endpoint at $s = 0$.*
    $z \sim \mathcal{N}(0, I)$ ▷ *Independent endpoint at $s = 1$.*
    $s \sim \text{Uniform}([0, 1])$ ▷ *Choose a time on this path.*
    $x_s \leftarrow (1 - s)x_0 + sz$ ▷ *Position at the chosen time.*
    $u \leftarrow z - x_0$ ▷ *Path velocity dx_s/ds.*

9: $\ell(\theta) \leftarrow \frac{1}{d}\|v_\theta(x_s, s) - u\|^2$ ▷ *Velocity-prediction loss.*
10: $g \leftarrow g + \frac{1}{B}\nabla_\theta \ell(\theta)$ ▷ *Add this example's gradient.*
11: **end**
12: $(\theta, q) \leftarrow U(\theta, q, g, \eta_j)$ ▷ *Update once per batch.*
13: **end**
14: **return** v_θ

Lines 4–8 construct a training example. Lines 4–5 draw the two endpoints independently; there is no prescribed pairing of a noise point with a data point. Line 6 samples a fresh time for each example, giving equal-length time intervals equal probability. Line 7 locates the point on the segment, and line 8 differentiates that segment with respect to time. The target velocity is constant along this particular segment. Both the input and its target can be computed directly, so making a training example requires no simulated trajectory.

Line 9 predicts velocity from position and time. The network receives only (x_s, s); the endpoints are used to construct its training target. It reuses the noise predictor's architecture with newly initialized weights and returns d velocity components. In `flow.toml`, continuous time uses Part 1′s sine/cosine features with $1000s$ replacing $t - 1$. These time features are concatenated with the coordinates in training and sampling. This scaling changes only the time features; the target in line 8 remains the derivative with respect to s. Training uses Part 1′s initialization, Adam optimizer and cosine learning-rate decay, with a new random seed.

Lines 9–12 fit the network. Line 9 averages the squared error over the d coordinates, with $d = 2$ here. Line 10 averages their gradients over B independent examples. The weights stay fixed while this batch is accumulated; line 12 updates them once. The learning rate decreases toward zero to make late updates smaller. The training-update counter j and example counter i organize optimization; s tells the network where the input lies in the data-to-noise evolution.

The velocity learned by line 9. Different pairs can pass through the same position at the same time with different velocities. The network cannot identify the pair from (x_s, s) alone. By Part 1′s squared-error identity, the optimal prediction is their conditional average, the marginal velocity:

$$v(x, s) = \mathbb{E}[u \mid x_s = x, s]. \tag{15}$$

Here "marginal" means averaging over the unobserved endpoints. This is the minimizer of the expected training loss over all prediction functions; a trained network approximates it. Although each training segment is straight, this average changes with position and time, so following it can give curved trajectories.

Probability conservation under the average velocity. Let $p_s(x)$ be the density of the positions constructed in line 7. Its *probability flux* is $p_s(x)v(x, s)$: density times the conditional average velocity. This is also the total flux of the individual training paths through each position. Consequently the density satisfies the *continuity equation*, expressing local conservation of probability:

$$\partial_s p_s(x) + \nabla_x \cdot (p_s(x)v(x, s)) = 0. \tag{16}$$

When the velocity defines a unique flow, its *ordinary differential equation* (ODE), $dx_s/ds = v(x_s, s)$, transports these same densities. Their endpoints are the data distribution at $s = 0$ and the standard Gaussian at $s = 1$. Integrating the exact field backward therefore transports Gaussian noise to the data distribution. The conditional average gives the required probability flow even though a generated trajectory need not follow an individual training segment.

Euler sampling with the learned velocity. Algorithm 5 uses the trained network to generate a new sample. Its Euler steps hold the predicted velocity constant over each time interval. Choose a decreasing time grid; the step-count comparison uses S equal intervals.

Algorithm 5. Generate a sample with the velocity network

Input: Fixed velocity predictor v_θ trained by Algorithm 4 on $\mathbb{R}^d$.
Decreasing times $1 = s_0 > s_1 > \ldots > s_S = 0$.
Output: Generated sample x_0.

1: $x_1 \sim \mathcal{N}(0, I)$ ▷ *The only random draw.*
2: **for** $k = 0, \ldots, S - 1$ **do**
3: $(s, s') \leftarrow (s_k, s_{k+1})$ ▷ *Current and next times.*

4: $\hat{v} \leftarrow v_\theta(x_s, s)$ ▷ *Velocity at the current state.*
5: $x_{s'} \leftarrow x_s - (s - s')\hat{v}$ ▷ *Step toward decreasing time.*
6: **end**
7: **return** x_0

Line 1 chooses the starting sample. Draw a fresh Gaussian point for each sample to be generated. The trained weights stay fixed throughout the loop; once the starting point is chosen, the path is deterministic.

Lines 3–5 follow the velocity backward in time. Line 4 performs a forward evaluation of the trained network at the current position and time. The network receives s, encoded as in training, while k only counts sampling steps. Its prediction describes motion toward increasing s, from data to noise. Line 5 subtracts that velocity times the positive interval length to move toward data. The next iteration evaluates the velocity again at the new position and time. Line 7 returns the endpoint at time zero. Network approximation and finite Euler steps both affect its distribution; the verification below measures their effect.

For the conservation proof and conditional training construction, see the Flow Matching Guide, Sections 3.5 and 4.2–4.5. The guide runs time from noise to data; here data is at $s = 0$.

Computing the exact mixture velocity. Within each Gaussian component, the conditional average velocity can be calculated exactly. At (x, s), weight these component velocities by the probabilities that the components produced $x_s = x$; these probabilities are their densities there divided by the sum of those densities. This gives the exact mixture velocity $v(x, s)$.

Skipping DDPM levels for the step-count comparison. The comparison also needs a random sampler on the coarser grid. For a jump from t to t', set $a = \overline{\alpha}_t / \overline{\alpha}_{t'}$. The forward jump shrinks by $\sqrt{a}$ and adds noise of variance $1 - a$. Use Part 1's reverse rule with these effective coefficients:

$$\mu_{t \to t'} = \frac{1}{\sqrt{a}} \left(x_t - \frac{1 - a}{\sqrt{1 - \overline{\alpha}_t}} \varepsilon_\theta(x_t, t) \right). \tag{17}$$

Add Gaussian noise of variance $1 - a$ when $t' > 0$, and none when $t' = 0$. For adjacent levels this is exactly Part 1's choice of variance β_t. The comparison uses evenly spaced levels before reaching zero.

KEY CONCEPTS
- **Flow matching and marginal velocity:** regression on straight-line velocities learns their conditional mean, which Euler steps follow from noise to data.
- **Probability conservation:** the averaged path velocities carry the same probability flux as the constructed training paths.

3 VERIFY

(1) **The learned velocity matches the exact marginal velocity.**

> `Plot the learned and exact velocity fields at s=0.1,0.5,0.9 on a grid over the samples, and report their relative RMS difference at each s, the RMS of the difference divided by the RMS of the exact velocity.`

The *relative root-mean-square* (*RMS*) error measures how large the prediction errors are compared with the exact arrows. For N test points x_i at a fixed s, it is

$$\sqrt{\frac{\sum_{i=1}^{N} \|v_\theta(x_i, s) - v(x_i, s)\|^2}{\sum_{i=1}^{N} \|v(x_i, s)\|^2}}. \tag{18}$$

Squaring before averaging prevents opposite errors from cancelling. The arrows overlap; yours should differ by less than 0.1 at every s, and the reference gives 0.011, 0.017 and 0.058. The arrows show dx_s/ds for increasing s; generation runs against them. Near $s = 1$ the field approaches $v(x, 1) = x$, because the mixture's mean is zero.

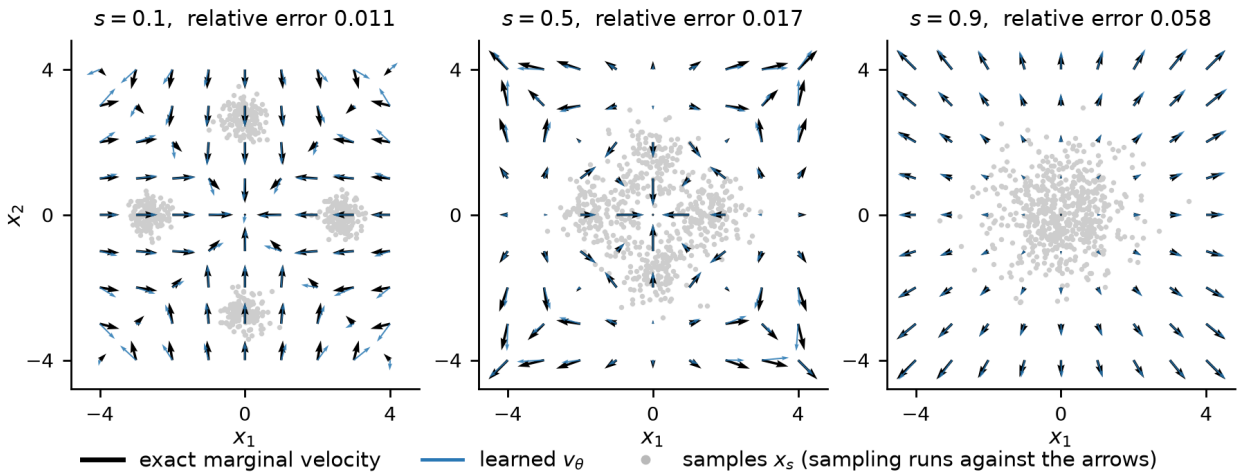


Figure 9: The learned field reproduces the mixture's marginal velocity, which is why Euler steps can follow it. Arrow lengths are clipped in the far field.

(2) **Flow matching and DDIM need tens of steps; DDPM needs more.**

> `Plot the energy distance, the largest cluster-fraction error and the largest width error against the number of steps for the three samplers, with the null scale shaded.`

The plot asks how many sampling steps are needed to approach the target distribution. The horizontal axis counts steps, each requiring one network evaluation. The energy-distance axis measures discrepancy between the whole generated and target distributions; smaller is better. By 20 steps your flow sampler should reach an energy distance below 0.005 and widths within 0.03 of 0.30. The reference flow sampler reaches 0.0015 at 20 steps, just above the null 95% quantile 0.0013; DDIM reaches 0.0038, and DDPM needs about 50 levels. On this mixture, flow matching's gain over DDIM is modest.

Energy distance compares whole distributions. Let X, X' be generated samples and Y, Y' target samples, all independent with finite expected norms. Define

$$D = \underbrace{2\mathbb{E}\|X - Y\|}_{\text{between distributions}} - \underbrace{\mathbb{E}\|X - X'\| + \mathbb{E}\|Y - Y'\|}_{\text{within distributions}}. \tag{19}$$

Even samples from the same distribution differ. Subtracting the two within-distribution terms removes this ordinary spread. The population distance satisfies $D \geq 0$, with $D = 0$ exactly when the distributions agree. See Székely and Rizzo, Section 2 for the proof.

The shaded bands show variation from finite sampling. Even two sets drawn from the exact target give different measured distances. The distances from the 20 pairs of independent exact sample sets establish the *null scale*. Their 95% *quantile*, the value below which 95% of those distances fall, sets the shaded comparison band for energy distance. Replacing the expectations by averages over distinct pairs can even give a negative estimate. A generated sample set within the null scale shows no discrepancy this comparison can distinguish at the chosen sample size. Cluster fractions also fluctuate: N independent draws do not place exactly a quarter of the points in each cluster. For a cluster of probability p, the fraction's *standard error*, its standard deviation across repeated sets, is $\sqrt{p(1-p)/N}$. The reference shades two standard errors for $p = 0.25$, the *binomial* sampling variation.

The coarsest grids fail even with exact predictors. One or two steps fail for every sampler, even with the exact noise prediction or velocity: the samples collapse toward the mean. For one DDIM step, Algorithm 3, line 6 goes directly to level zero and returns the clean estimate from line 5. Even with an exact predictor, that estimate averages over the clean points compatible with the starting noise. At this high noise level, all four clusters remain plausible, so their average lies near the centre instead of inside a cluster. Intermediate steps change the state and recompute the prediction, gradually moving samples toward individual clusters. Replacing the network by an exact predictor removes learning error; the remaining failure comes from the sampling steps.

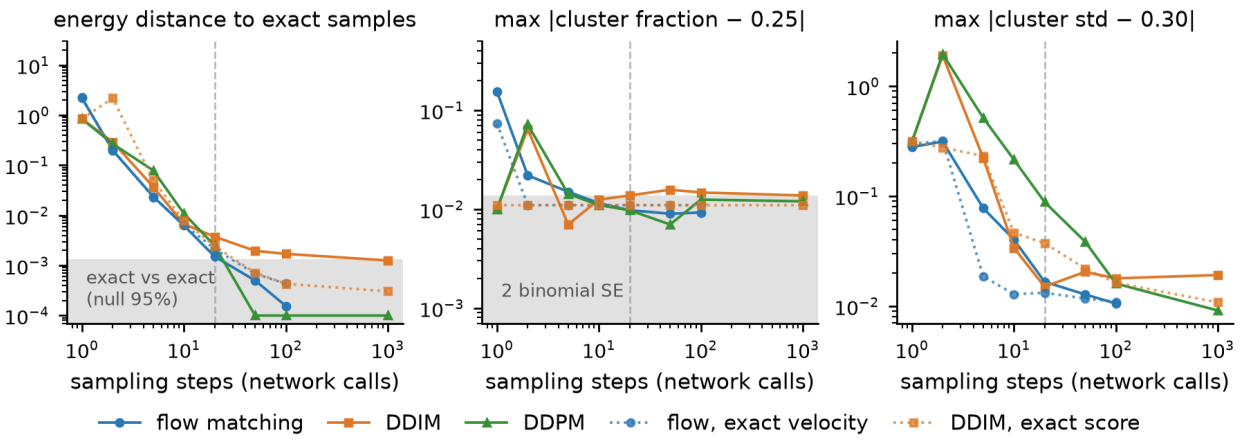


Figure 10: Flow matching and DDIM reach the target in tens of steps while DDPM needs about fifty. Shaded bands are the exact-versus-exact null scale and two binomial standard errors of a cluster fraction; energy distances below 10^{-4} are drawn at 10^{-4}. The dashed line marks 20 steps.

Part 4: The same predictor on photographs

Goal: load a predictor already trained on CIFAR-10 photographs and generate recognizable images with Part 1's DDPM update, without further training. Part 5 uses the same model to fill in missing pixels.

(1) DO

Download the CIFAR-10 noise predictor, already trained, from Hugging Face. Keep these files for Part 5.

```
uv run --with huggingface-hub==1.33.0 hf download google/ddpm-cifar10-32 \
  config.json diffusion_pytorch_model.safetensors \
  --revision 267b167dc01f0e4e61923ea244e8b988f84deb80 \
  --local-dir inputs/cifar-model
```

Generating four images takes roughly three minutes on the reference CPU.

```
Load the downloaded pretrained weights and configuration as a fixed noise predictor in
JAX. Match the network in this reference implementation:
https://github.com/huggingface/diffusers/tree/v0.40.0
```

```
Use the pretrained predictor with the same 1000-step DDPM sampler as on the mixture,
without clipping during sampling. Use float32 arrays and pass noise level t as model
index t-1. Keep the downloaded weights fixed.
```

(2) UNDERSTAND

Loading the predictor from weights and configuration. A pretrained network has already learned its weights. We use `google/ddpm-cifar10-32`, trained to predict noise in photographs. It is *unconditional*: it receives no requested object category or text. The weights file supplies its parameters; the *configuration* file specifies the architecture. The download's *revision* fixes the version of both. We keep those weights fixed throughout sampling.

Predicting one noise value per pixel and colour channel. Each pixel has red, green and blue values, its three *colour channels*. The image is an *array*, an indexed collection of numbers; its *shape* counts entries along each direction. For a batch of B images the predictor maps

$$\varepsilon_\theta : (x_t, t) \mapsto \hat{\varepsilon}, \quad x_t, \hat{\varepsilon} \in \mathbb{R}^{B\times 3\times 32\times 32}. \tag{20}$$

Channels come before height and width. The output estimates one noise value per pixel and channel, exactly the quantity Part 1's DDPM sampler needs.

Matching the checkpoint's indexing and colour range. Apply the same 1000-step DDPM algorithm to these arrays. Use *float32*, storing each number approximately with 32 binary digits. The checkpoint counts levels from zero: pass our

t as model *index* $t - 1$. Clean colours lie in $[-1, 1]$; for display map x to $\frac{x+1}{2}$ and clip to $[0, 1]$. Keep the sampling states unclipped.

Convolutions extract local image features. The image predictor uses *convolutions*: each slides a learned *filter* over the image. At each position it multiplies nearby values in all input channels by matching weights, sums the products and adds a learned bias. Each filter produces one *feature channel*, a grid of responses.

The U-Net combines coarse context with pixel detail. The *U-Net* [1] reduces the grid size, or spatial *resolution*, to gather wider context, then enlarges it to predict noise at every original pixel. Its down path reduces side lengths from 32 through 16 and 8 to 4; the up path returns to 32. *Skip connections* append saved down-path features to up-path features at matching resolutions, preserving detail. The final convolution returns three noise channels. The configuration fixes the layer dimensions and operations that the implementation must match. As in Part 1, *time features* computed from t are supplied throughout the network; here they are called the *time embedding*.

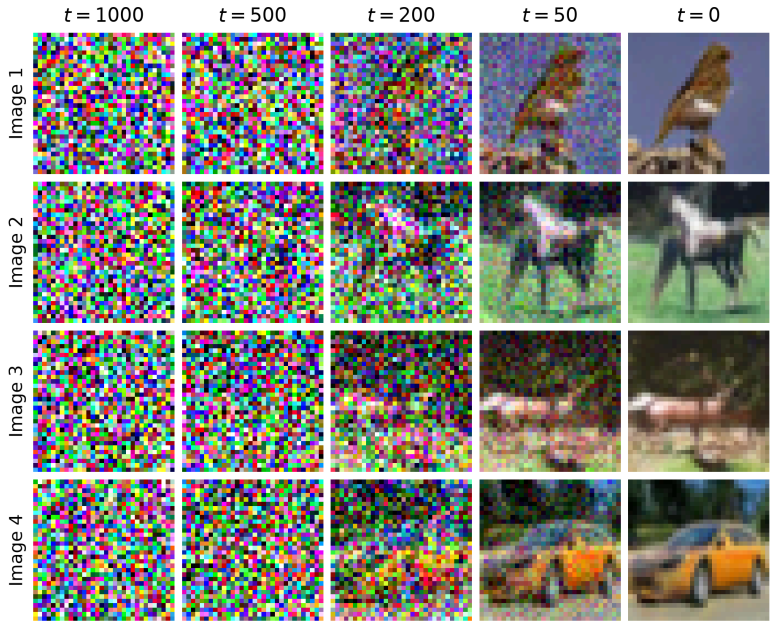
KEY CONCEPTS
- **Weights and configuration:** learned parameter values and the architecture needed to use them.
- **Noise prediction:** noisy image and noise level in, an image-shaped noise estimate out.
- **Generation:** Part 1′s DDPM update, unchanged, turns that estimate into an image.

3 VERIFY

(1) **The downloaded predictor generates recognizable photographs.**

```
Generate four images from Gaussian noise using the downloaded predictor and the
1000-step DDPM sampler. Visualize stages of the reverse process and the final
images. Compare them with real CIFAR-10 images using the Week 6 image-gallery
scripts.
```

Follow each noise image through the reverse process and compare the emerging colours, shapes and objects with the CIFAR-10 examples of Week 6′s Part 1. Structure appears late: at $t = 200$ the images are still mostly noise, and objects become recognizable only between $t = 200$ and $t = 50$. Explain which part of this process was learned before the download and which part your sampler computes now.



Pretrained noise predictor; the same DDPM update, all 1000 steps.
Recorded states; display clips to [−1, 1]. Native 32 × 32 pixels.

Figure 11: The downloaded weights and Part 1′s update produce recognizable objects without further training. Each row follows one sample; states are clipped only for display.

Part 5: Filling in missing pixels with the downloaded model

Goal: use the downloaded predictor to fill in a hidden square of a photograph, and check that every filled image keeps the visible pixels exactly. Repeating the process with different random choices should fill the hole with different, plausible content.

① DO

Fetch the prepared CIFAR-10 images, stored in $[-1, 1]$, into your `week7/` directory. The file holds four test images.

```
mkdir -p inputs
curl -fL https://giggleliu.github.io/AMAT5315-2026Fall/downloads/week7-images.npz \
  -o inputs/images.npz
```

```
With the pretrained CIFAR-10 predictor, inpaint images from observed pixels and a
mask, given the observations and the mask only. Implement RePaint with 50 reverse
jumps over levels 1000,980,...,20,0, two attempts per jump except the last, forward
noise between attempts, and known pixels replaced by forward-noised observations.
```

```
Use the four test images in inputs/images.npz. Hide rows and columns 6:26, all three
channels, and inpaint each image with RePaint using seeds 7201,7202,7203,7204. Record
each completion and its hidden-pixel mean squared error (MSE).
```

② UNDERSTAND

Specifying visible and missing pixels with a mask. Filling the missing part of an image is inpainting. The filled image is a completion. The visible values are the *observations*; the sampler does not receive the hidden truth. A mask M with the image's shape specifies which values may change. It contains 1 at observed entries and 0 at hidden ones. Our mask hides a central 20×20 square in all three colour channels. Write Y for the observed image values where $M = 1$; set its unused entries to zero where $M = 0$.

Noising observations to match the sampling state. Before inserting observations into a state at level t', Algorithm 6, line 21 applies Part 1's direct noising formula to Y. These are the forward-noised observations, $Y_{t'}$.

Keeping observed pixels and proposing hidden pixels. RePaint is an inpainting sampler that replaces the observed pixels after every reverse step and can retry steps to improve the fit across the hole's boundary. Call the image suggested by a reverse step its *proposal*, $Z_{t'}$. Keep its hidden pixels, but replace its observed pixels with $Y_{t'}$. This combination is the *merge* in Algorithm 6, line 22; $\odot$ multiplies matching entries. Both regions must be at the same noise level, which is why we use $Y_{t'}$.

Drawing RePaint's proposal across skipped levels. To reduce the number of network evaluations, RePaint visits $1000, 980, \ldots, 20, 0$. Each proposal jumps from t to $t' < t$, skipping levels as in Part 3's DDPM comparison. We use Part 2's Tweedie estimate $\hat{X}_0$, clipped to the clean image range $[-1, 1]$. Only this estimate is clipped, not the noisy state. Set $a = \frac{\overline{\alpha}_t}{\overline{\alpha}_{t'}}$ as in Part 3. If we treat the clean estimate as the original image, the Gaussian forward rule gives a conditional distribution for the lower-level state. The box gives its mean μ and per-coordinate variance v. This variance differs from Part 3's choice $1 - a$ for the DDPM comparison; at $t' = 0$ it is zero.

Repeating reverse jumps in the RePaint loop. Use two attempts for each jump except the final jump to zero, which is taken once. The cost is $49 \times 2 + 1 = 99$ network evaluations. Every Gaussian draw below is fresh, independent of earlier draws, and has the image's shape. A *random seed* selects a sequence of random draws. As in Part 4, the predictor maps mathematical level t to checkpoint index $t - 1$. The clipping operations min and max act on each entry; $[\ell, u]$ denotes the clean image range.

Algorithm 6. RePaint inpainting

Input: Fixed image noise predictor ε_θ; observations Y and mask M; training noise variances $\beta_1, \ldots, \beta_T$; levels $T = t_0 > \ldots > t_S = 0$; attempts R_k per jump, with $R_S = 1$; clean-value bounds $[\ell, u]$.
Output: Completion X_0 with $M \odot X_0 = M \odot Y$.

1: $\overline{\alpha}_0 \leftarrow 1$

2: **for** $t = 1, \ldots, T$ **do**
3: $\quad \overline{\alpha}_t \leftarrow \overline{\alpha}_{t-1}(1 - \beta_t)$
4: **end**
5: $X_T \sim \mathcal{N}(0, I)$
6: **for** $k = 1, \ldots, S$ **do**
7: $\quad (t, t') \leftarrow (t_{k-1}, t_k)$
8: $\quad a \leftarrow \dfrac{\overline{\alpha}_t}{\overline{\alpha}_{t'}}$
9: $\quad$ **for** $r = 1, \ldots, R_k$ **do**
10: $\quad\quad$ **if** $r > 1$ **then**
11: $\quad\quad\quad f \sim \mathcal{N}(0, I)$
12: $\quad\quad\quad X_t \leftarrow \sqrt{a} X_{t'} + \sqrt{1 - a} f$
13: $\quad\quad$ **end**
14: $\quad\quad \hat{\varepsilon} \leftarrow \varepsilon_\theta(X_t, t)$
15: $\quad\quad \hat{X}_0 \leftarrow \dfrac{X_t - \sqrt{1 - \overline{\alpha}_t}\hat{\varepsilon}}{\sqrt{\overline{\alpha}_t}}$
16: $\quad\quad \hat{X}_0 \leftarrow \min\big(u, \max\big(\ell, \hat{X}_0\big)\big)$
17: $\quad\quad \mu \leftarrow \dfrac{\sqrt{\overline{\alpha}_{t'}}(1 - a)}{1 - \overline{\alpha}_t}\hat{X}_0 + \dfrac{\sqrt{a}(1 - \overline{\alpha}_{t'})}{1 - \overline{\alpha}_t} X_t$
18: $\quad\quad v \leftarrow \dfrac{(1 - \overline{\alpha}_{t'})(1 - a)}{1 - \overline{\alpha}_t}$
19: $\quad\quad \xi \sim \mathcal{N}(0, I), \quad \eta \sim \mathcal{N}(0, I)$
20: $\quad\quad Z_{t'} \leftarrow \mu + \sqrt{v}\xi$
21: $\quad\quad Y_{t'} \leftarrow \sqrt{\overline{\alpha}_{t'}} Y + \sqrt{1 - \overline{\alpha}_{t'}}\eta$
22: $\quad\quad X_{t'} \leftarrow M \odot Y_{t'} + (1 - M) \odot Z_{t'}$
23: $\quad$ **end**
24: **end**
25: **return** X_0

Why retrying lets hidden pixels respond to observations. The first proposal uses the state before the observations are merged. Adding forward noise to the merged state and repeating the jump lets the predictor respond to those observations when proposing the hidden pixels.

See Lugmayr et al., *RePaint*, Section 4.1, for the resampling method.

KEY CONCEPTS

- **Mask and inpainting:** the mask marks observed pixels; a completion keeps them and fills the rest.
- **Forward-noised observations:** the observed pixels, noised to the sampler's current level with Part 1's jump formula.
- **RePaint:** a coarse reverse sampler with the observed pixels overwritten after every jump; retry each jump after forward noising, except the final jump to zero.

3 VERIFY

(1) **Completions keep the observed pixels and disagree inside the hole.**

```
For each test image, show the truth, the observation, the four completions and the
pixelwise variance across them. Print each completion's hidden-pixel MSE and the
largest change of an observed pixel.
```

Every observed pixel should change by less than 10^{-5}. At $t' = 0$, Algorithm 6, line 21 returns the clean observations; line 22 inserts them unchanged. Outside the hole the four completions are identical; inside it they differ.

The *pixelwise variance* measures this disagreement across completions. The *hidden-pixel MSE* measures error against the withheld truth, averaging squared errors over the hole's 400 pixels and three channels. In the reference the hidden-pixel MSE averages 0.24 over images and seeds; on the ship below it ranges from 0.13 to 0.38 across seeds. Locate the disagreements on the variance map. Many images can agree with the visible pixels, so a plausible completion can still differ from the hidden truth and have a large MSE. Variation across seeds measures disagreement among completions; it does not measure their error against that truth.

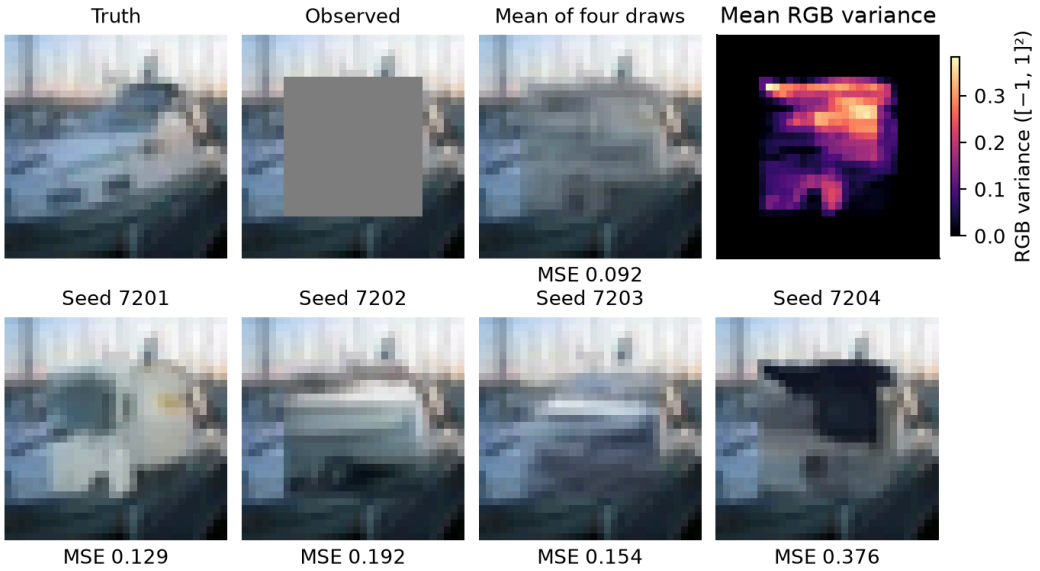


Figure 12: The observation leaves the hidden square undetermined, so each seed completes it differently. The reference ship image: truth, observation, the mean and pixelwise variance of four RePaint completions, and the four completions with their hidden-pixel MSE.

Save the results and prepare to show your work

```
Update README.md with each evidence figure beside its producing command. Explain the
transport comparison, the generated photographs and the inpainting result. Record
training settings, environment setup and the pinned model download. Keep downloaded
weights out of git. Push the source, environment and evidence.
```

Show the completed evidence at the start of Week 8. Nothing is submitted.

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.